



# AI Agents at Work

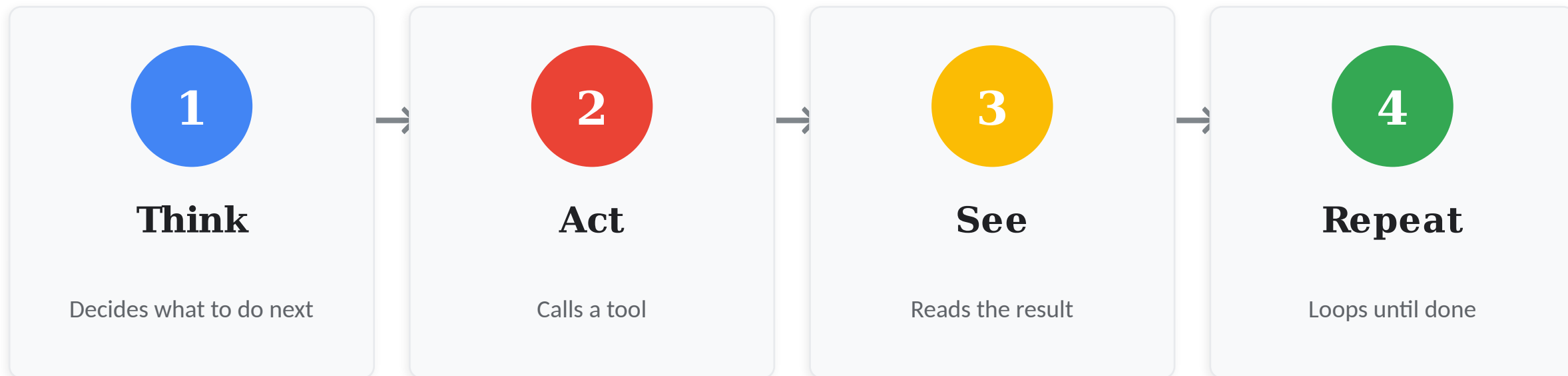
Tools • MCP • Skills • Memory

*You already know HOW agents think. Today, you put them to work — and meet the whole family.*

Computer Science • Hands-on with opencode + OpenRouter

# An Agent is an LLM in a Loop

You learned this already — the model uses tools, over and over, until the job is done. Four beats:



*The loop is the same everywhere. What changes is the TOOLS, the MEMORY, and the app around it.*

# There's a Whole Family of Agents

That same loop powers a fast-growing family of real tools. They differ in WHERE they run and WHAT they're best at — but under the hood, they're all loops with tools.



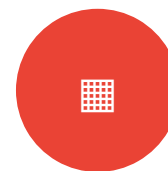
## Coding agents

Live in your code editor or terminal and build software



## Assistant agents

Run in the background and reach you on chat apps



## Agent platforms

Whole apps where many agents work at once

# Famous Agents in 2026

|                           |                      |                                                                  |
|---------------------------|----------------------|------------------------------------------------------------------|
| <b>Claude Code</b>        | <i>Anthropic</i>     | Terminal coding agent — great at big, careful code changes       |
| <b>Codex CLI</b>          | <i>OpenAI</i>        | Terminal coding agent — fast and great at automatic tasks        |
| <b>Google Antigravity</b> | <i>Google</i>        | A whole platform where many agents plan, build & test an app     |
| <b>OpenClaw</b>           | <i>Open-source</i>   | A personal assistant agent you run yourself; reaches you on chat |
| <b>Hermes Agent</b>       | <i>Nous Research</i> | Assistant agent that remembers and writes its OWN new skills     |
| <b>opencode</b>           | <i>Open-source</i>   | The friendly agent WE'll use today — works with any model        |

# Same Recipe, Different Flavors

It looks like a lot to learn — but every one of these is built from the SAME four ingredients. Learn them once, and you understand them all.



## **A Model**

The brain that thinks



## **Tools**

Hands to act in the world



## **Memory**

Notes so it doesn't forget



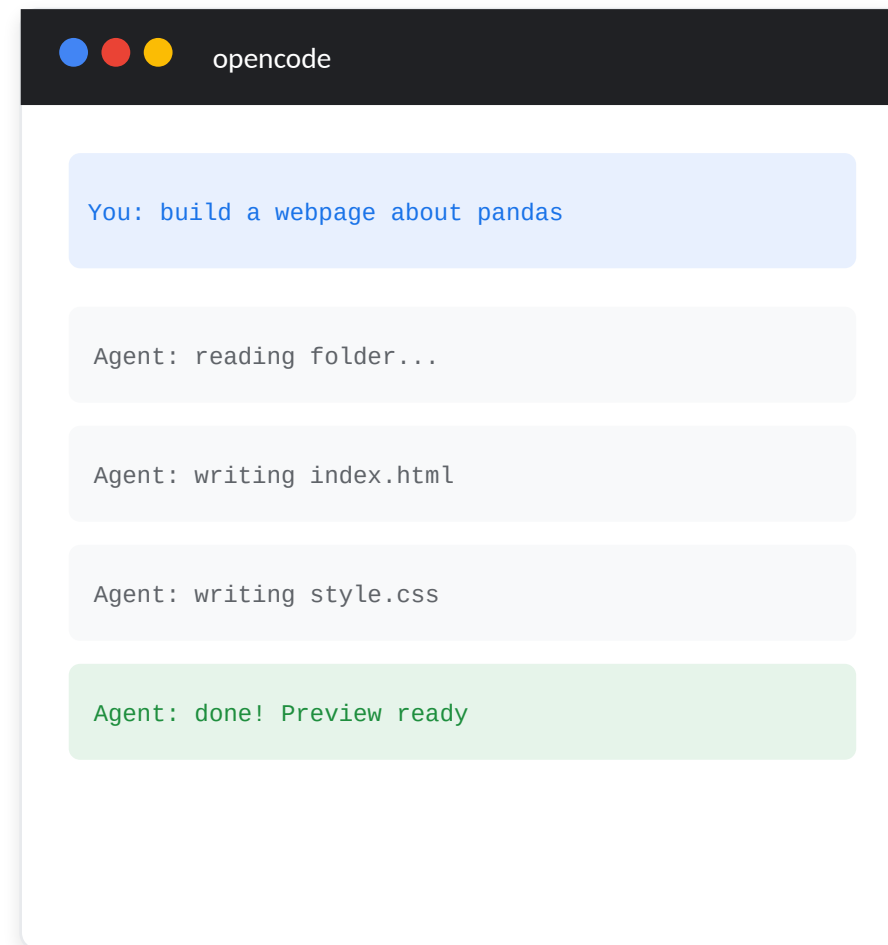
## **A Loop**

Repeats until the job's done

# Meet opencode

opencode is a free, open-source agent you run on your own computer. We'll use the desktop app — a friendly window, no scary terminal — connected to DeepSeek V4 Flash through OpenRouter.

- ✓ **Runs real agents**
- ✓ **Works with any model**
- ✓ **Supports MCP + Skills**
- ✓ **Asks before big actions**



# Three Ways to Give an Agent Tools

## 1 Built-in Tools

*Come with the agent*

Read & write files, run commands

## 2 MCP Servers

*Plug-in tool packs*

Snap on NEW powers from outside

## 3 Skills

*Saved instructions*

Teach it to do a task YOUR way

*We'll look at each one up close.*

# Built-in Tools

These come with opencode. They're how the agent touches your computer — the same file & command ideas from our terminal lessons!



Open a file



Create a file



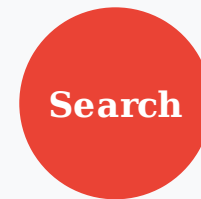
Change a file



Run a command



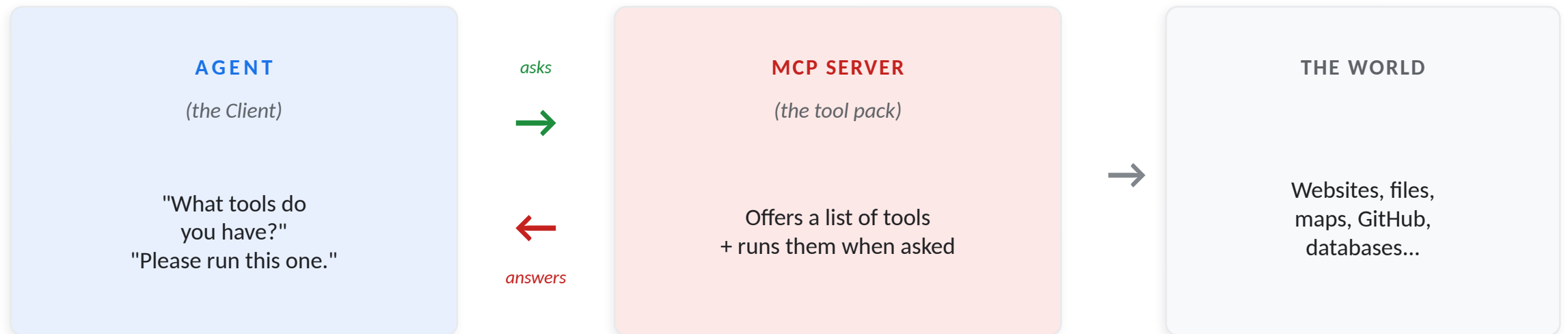
See a folder



Find text

# How MCP Actually Works

MCP = Model Context Protocol. "Protocol" just means an agreed way to talk — like how every USB plug fits every USB port. The agent is the CLIENT; a tool pack is the SERVER.



*The big idea: any agent can use any MCP server. Build a tool pack once — every agent can plug it in.*

# Plug-In Superpowers

Each MCP server adds a whole set of new tools. A few real ones:



## Web Fetch

Read live web pages



## Filesystem

Work across a whole folder



## Maps

Look up places & directions



## GitHub

Read & manage real code

# How Skills Work

A skill is a little file you write. The agent reads its description, and WHEN your request matches, it follows your steps — every time, your way.

## Every skill has 3 parts

- 1 Name** what it's called
- 2 Description** WHEN the agent should use it
- 3 Steps** what to do, in order

## Example: a "quiz-maker" skill

```
name: quiz-maker

description: Use when I
ask for a quiz on a topic.

steps:

1. Ask for the topic
2. Write 5 questions
3. Save as quiz.html
4. Add an answer key
```

# MCP vs. Skills

## MCP Server

*Gives the agent NEW abilities*

- Adds tools it didn't have
- Often connects to the outside world
- Usually built by developers
- Like: "here are new hands"

## Skill

*Teaches the agent HOW you want a job done*

- Uses tools it already has
- Just saved instructions & steps
- YOU can write one easily
- Like: "here's a recipe to follow"

# How Agent Memory Works

By itself, a model forgets everything between chats. Memory is how an agent REMEMBERS — so it can pick up where it left off and learn your preferences.

## Short-term

The current conversation — what's happened so far this session

## Working notes

A scratchpad / to-do list the agent keeps while doing a big job

## Long-term

Saved facts in a file or database it can look up later, across sessions

**Cool example:** Hermes Agent finishes a hard task, then writes itself a NEW skill so it remembers how next time. Memory + skills working together!

# Give the Agent a Good Mission

Agents work best with a clear goal — not a vague wish. Same specificity skill from last class!

## WEAK MISSION

"make something cool"

The agent has to guess what you mean, the format, and when it's done.

## STRONG MISSION

"Make a webpage about red pandas. Include 3 fun facts, a title, and a green background. Save it as pandas.html."

Clear goal, clear features, clear finish line. The agent nails it.

# Read the Loop as it Runs

When your agent works, narrate what it's doing with your group. Match each line to a beat of the loop:

Agent: "I'll make the file first."

THINK

Calls Write tool -> creates pandas.html

ACT

Tool result: file saved

SEE

Agent: "Now I'll add the fun facts."

THINK -> REPEAT

*The agent isn't magic — it's steps you can follow.*

# Today's Missions



## First Contact

Give your agent a mission and watch it build



## Read the Loop

Narrate each tool call as a team



## Teach a Skill

Write saved instructions the agent follows



## Build Challenge

Teams race to build something awesome

# Agent Rules of the Road



## Do

- Read what the agent wants to do before approving
- Give clear missions with a finish line
- Work in a fresh, empty folder
- Test what it makes — agents make mistakes



## Don't

- Never share or post your API key
- Don't approve commands you don't understand
- Don't put private info in your prompts
- Don't let it delete files you care about



# Go Put an Agent to Work

Model + Tools + Memory + a Loop = an agent. Now go direct one.

Head to the classwork and work through Missions A–D with your group. Watch the loop, approve each step, and have fun building. Kahoot at the end!

*You're not just using agents — you're directing them.*